

Improving Genetic Programming with Behavioral Consistency Measure

Krzysztof Krawiec¹ and Armando Solar-Lezama²

¹ Institute of Computing Science, Poznan University of Technology, Poznań, Poland

`krawiec@cs.put.poznan.pl`

² Computer Science and Artificial Intelligence Laboratory,
Massachusetts Institute of Technology, Cambridge, MA, USA

`asolar@csail.mit.edu`

Abstract. Program synthesis tasks usually specify only the desired output of a program and do not state any expectations about its internal behavior. The intermediate execution states reached by a running program can be nonetheless deemed as more or less preferred according to their information content with respect to the desired output. In this paper, a consistency measure is proposed that implements this observation. When used as an additional search objective in a typical genetic programming setting, this measure improves the success rate on a suite of 35 benchmarks in a statistically significant way.

Keywords: Program synthesis, genetic programming, entropy, multi-objective search.

1 Introduction

One of the main challenges for genetic programming (GP)—or for that matter, for any approach to program synthesis based on explicit search over a space of programs—is to decide on a fitness function that captures the relative quality of different proposed solutions. A common approach is to consider the quality of the *output* of a given program on a set of candidate inputs, possibly augmented with some structural constraints to prevent the search algorithm from overfitting to the training inputs (examples). The problem with this approach is that there is a fundamental mismatch between the search approach, which operates on the *structure* of the program, and the fitness function which is based on its input/output *behavior*. An ideal fitness function would instead be one that rewards programs that are structurally close to a correct solution: a program that is only a few small modifications away from being correct is better than one that has to be completely transformed in order to work, even if the former produces incorrect output on every input. The problem, of course, is that we do not know how the correct program looks like—if we did, we would not be searching for it—so we are left with behavior-based measures of correctness.

In this paper, we propose a new fitness measure that tries to better capture how close a proposed solution is from being correct by assessing the quality of intermediate values of the program in addition to the quality of the output.

At first glance, it may seem that such a measure would run into the same problem as any other structural quality measure: since we do not know what the correct solution looks like, we also do not know what the intermediate values produced in the process of that computation should be. However, we do know that these intermediate values must preserve certain information about the input that is necessary to produce the desired output. Our main contribution is to show that information theoretic measures of the quality of intermediate values can improve upon traditional output-based measures in a statistically significant way.

The intuition for the approach is as follows: if two inputs are supposed to produce different outputs, then any program prefix that loses the information necessary to distinguish between these two inputs is doomed to failure since once lost, the information cannot be recovered. By contrast, if two inputs are supposed to produce the same output, then a program prefix that produces the same value for these two inputs will eliminate the risk that the rest of the program incorrectly assigns different values to these two inputs. The rest of the paper formalizes this intuition and exploits it in a new fitness measure based on *information consistency*. We implement the approach in a single-objective and a multi-objective variant, and evaluate it on 35 benchmarks representing three different domains. The experimental results clearly suggest that the involvement of information consistency almost always increases the likelihood of synthesizing the correct program.

2 Information Consistency

Conceptual Background. The proposed approach works by analyzing the internal behavior of programs, by which we mean the effects of computation conducted at intermediate stages of program execution. Conceptually, we examine such effects by placing *traps* (breakpoints) at selected locations in program code and inspecting the program state when execution reaches these traps. We assume that traps are not embraced by loops or conditional statements, thus ensuring that a deterministic program always visits all traps in the same order *independently of program input*.

When stopped at a trap, an executing program produces a certain *state* of the execution environment, which we assume to depend only on the part of the program that is syntactically associated with the trap (or, informally speaking, ‘scoped’ by the trap). Depending on the adopted programming paradigm, the particular interpretation of these notions will vary. For register-based sequential programs, a state would be the contents of all registers, and it would depend on the entire code executed so far (i.e., program prefix ending at the trap). For purely functional programming, by state we would mean the value calculated in the corresponding function call, caught in the process of being passed to the caller. In this study we consider functional tree-based GP and will place traps at tree nodes, so a state will be the value returned by the corresponding subtree.

Let s_i^k denote the state of the program at the k th trap when executed on the i th training example (input data). The sequence of the states $s_i^1, s_i^2, \dots$ traversed